

Material de Aula

Curso: Bacharelado em Ciência da Computação

Disciplina: Tecnologia de Orientação à Objetos (TOO)

Professora: Vanessa Lago Machado

Parte 2: Programação Orientada à Objetos (POO)

7. Pilar Herança: Reutilizando e Especializando Código

Após compreendermos os conceitos de Classes e Objetos, avançamos para um dos quatro pilares essenciais da Programação Orientada a Objetos: a

Herança: A herança é o mecanismo que permite a uma nova classe (chamada de subclasse) herdar atributos e métodos de uma classe já existente (a superclasse).

Esse pilar promove a reutilização de código e a criação de hierarquias de classes que representam relações do tipo “**é um tipo de**”.

7.1. Conceitos Fundamentais: Superclasse e Subclasse

Na herança, trabalhamos com dois tipos de classes:

- **Superclasse (Classe Pai):** É a classe que cede seus atributos e métodos. Ela contém as características e comportamentos mais genéricos. Por exemplo:

`Pessoa`, `Animal`, `Conta_Bancaria`.

- **Subclasse (Classe Filha):** É a classe que herda da superclasse. Ela recebe todos os atributos e métodos da classe pai e pode adicionar seus próprios comportamentos e características específicas ou modificar os herdados. Por exemplo,

`Pessoa_Fisica` é um tipo de `Pessoa`.

7.2. Herança na Prática em Python

A sintaxe para criar uma subclasse em Python é simples: basta colocar o nome da superclasse entre parênteses após o nome da subclasse.

Vamos modelar um sistema de cadastro onde temos `Pessoa` (conceito genérico) e `Pessoa_Fisica` (conceito específico que herda de `Pessoa`).

Exemplo Prático:

```
# 1. Definição da Superclasse (Classe Pai)
class Pessoa:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email

    def exibir_dados(self):
        return f"Nome: {self.nome}, E-mail: {self.email}"

# 2. Definição da Subclasse (Classe Filha)
# Pessoa_Fisica herda de Pessoa
class Pessoa_Fisica(Pessoa):
    def __init__(self, nome, email, cpf):
        # Chama o construtor da classe pai (Pessoa) para inicializar nome e email
        super().__init__(nome, email)
        # Adiciona um atributo específico da subclasse
        self.cpf = cpf

# 3. Utilização dos Objetos
pessoa_comum = Pessoa("Ana", "ana@email.com")
pessoa_fisica = Pessoa_Fisica("Carlos", "carlos@email.com", "123.456.789-00")

print("--- Dados da Pessoa Comum ---")
print(pessoa_comum.exibir_dados())

print("\n--- Dados da Pessoa Física ---")
# A subclasse pode usar o método 'exibir_dados' herdado da superclasse
print(pessoa_fisica.exibir_dados())
print(f"CPF: {pessoa_fisica.cpf}")
```

7.3. Estendendo Funcionalidades com `super()`

Como visto no exemplo acima, a função `super()` é usada para chamar métodos da superclasse a partir da subclasse. Isso é extremamente útil para estender funcionalidades sem reescrever código. No nosso exemplo, o `__init__` de `Pessoa_Fisica` usa `super().__init__(nome, email)` para executar a lógica de inicialização da classe `Pessoa` antes de adicionar seus próprios atributos.

Podemos também estender outros métodos. Por exemplo, poderíamos reescrever `exibir_dados` em `Pessoa_Fisica` para incluir o CPF:

```
class Pessoa_Fisica(Pessoa):
    def __init__(self, nome, email, cpf):
        super().__init__(nome, email)
        self.cpf = cpf

    # Reescrevendo e estendendo o método da classe pai
    def exibir_dados(self):
        # Chama a implementação original da superclasse
        dados_pai = super().exibir_dados()
        # Adiciona a informação específica da subclasse
        return f"{dados_pai}, CPF: {self.cpf}"

# Testando a nova implementação
pessoa_fisica = Pessoa_Fisica("Carlos", "carlos@email.com", "123.456.789-00")
print(pessoa_fisica.exibir_dados())
# Saída: Nome: Carlos, E-mail: carlos@email.com, CPF: 123.456.789-00
```

7.4. Vantagens da Herança

Utilizar herança traz diversos benefícios para a arquitetura do software:

- **Reutilização de Código:** Evita a duplicação de código, já que atributos e métodos comuns são definidos uma única vez na superclasse.
- **Facilidade de Manutenção:** Qualquer modificação ou correção na superclasse é automaticamente herdada por todas as suas subclasses.
- **Abstração:** Permite a criação de classes genéricas que definem um “contrato” ou uma base, que pode então ser especializada por subclasses.

- **Polimorfismo:** A herança é a base para o polimorfismo (“muitas formas”). Isso permite que um objeto de uma subclasse seja tratado como se fosse um objeto da superclasse, possibilitando a criação de código mais flexível e genérico.

—

